tabpfn_v3_classification_tutorial

July 5, 2026

1 TabPFN v3 — Zero to Mastery: Classification on a Heavy, Complex Dataset

A complete, hands-on tutorial on TabPFN v3, Prior Labs' prior-fitted-network tabular foundation model, applied end-to-end to a genuinely heavy, multi-class classification problem: predicting one of 7 forest cover types for 580,000+ real Colorado land patches.

This is one of three companion notebooks (alongside Mitra and XGBoost) that all train and evaluate on the **same dataset and the same held-out evaluation set**, so their results are directly comparable even though each notebook is fully standalone.

Estimated runtime: ~10–20 minutes. TabPFN is **GPU-recommended** — its own documentation states CPU is only feasible for datasets under ~1,000 rows, so we check for a GPU and size our context accordingly.

License note: TabPFN-3's pretrained weights are non-commercial licensed (Prior Labs License).

1.1 1. What Is TabPFN v3?

TabPFN ("**Tab**ular **P**rior-**F**itted **N**etwork") comes from Prior Labs (Hollmann, Müller, Purucker, Hutter, and collaborators), and is the model that originated the whole lineage of in-context tabular foundation models this notebook series covers — TabPFN's 2023 ICLR paper and its 2025 Nature paper predate TabICL and TabFM, both of which explicitly build on ideas TabPFN introduced.

The core idea, unchanged since the original paper: a transformer is pretrained once on an enormous number of *synthetic* datasets sampled from a prior over plausible data-generating processes. At inference time, it reads your real training data as context and approximates Bayesian posterior inference over labels for new points, in a single forward pass — no gradient training on your data at all.

What's new in v3 (the current default in the `tabpfn` package as of this writing) is scale. Per Prior Labs' own documentation, each TabPFN generation raised its practical size ceiling:

Checkpoint	Recommended envelope
TabPFN-2.6 (previous default)	up to 100,000 rows, 2,000 features
TabPFN-3 (current default)	up to **1,000,000 × 200**, **100,000 × 2,000**, or **1,000 × 20,000** (rows × features — larger feature counts trade off against row capacity)

Native categorical handling is also built in — you don't need to one-hot encode or manually specify which columns are categorical for basic use.

Real constraint, from Prior Labs' own README: *"TabPFN is slow to execute on a CPU… on CPU, only small datasets (1,000 samples) are feasible."* GPU is not a nice-to-have here, it's close to a requirement for anything beyond toy-sized data — we check for one explicitly in Section 3.

Source: github.com/priorlabs/tabpfn — `pip install tabpfn` (version verified for this notebook: **8.0.8**) — TabPFN-2.5 technical report (arXiv:2511.08667) — original Nature paper.

1.2 2. Environment Setup

uv — `uv venv --python 3.13.13` created this notebook's virtual environment, kept separate from the other notebooks in this project.

[ ]:
```
import sys

!uv pip install --python {sys.executable} -q tabpfn scikit-learn pandas numpy␣
↪matplotlib seaborn
```

[ ]:
```
import time
import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, log_loss,␣
↪balanced_accuracy_score

sns.set_theme(style="whitegrid")
print(f"torch {torch.__version__} | cuda available: {torch.cuda.
↪is_available()}")
```

1.2.1 Device Check — Not Optional Here

Unlike the other models in this notebook series, TabPFN's own documentation is explicit that CPU is only realistic for tiny datasets. We check for a genuinely free GPU and size our context to what fits comfortably; if no GPU is available, this notebook will still run, but with a much smaller context to stay within a reasonable runtime.

[ ]:
```
def pick_device(min_free_gb=4.0):
    if not torch.cuda.is_available():
        return "cpu"
    free_bytes, _ = torch.cuda.mem_get_info()
    free_gb = free_bytes / 1e9
```

```
    print(f"GPU free memory: {free_gb:.2f} GB")
    return "cuda" if free_gb >= min_free_gb else "cpu"


DEVICE = pick_device()
print("Selected device:", DEVICE)
```

1.3 Dataset: Forest Cover Type (Covertype) — 7-Class Classification

Our task is to predict **which of 7 forest cover types** grows on a 30x30m patch of land in the Roosevelt National Forest, Colorado, from cartographic variables — no remote sensing imagery, only elevation, slope, distance-to-water/roads/fire-points, and soil/wilderness classification. This is a real, well-established UCI benchmark, heavy and complex in exactly the ways this notebook series wants to stress-test:

- **581,012 rows** — large enough that a foundation model's context-size limits and a GBM's full-data training both genuinely matter.
- **7-class, meaningfully imbalanced** target (from 211,840 rows of the majority class down to just 2,747 of the rarest) — a real step up in complexity from a binary classification problem.
- **A one-hot-to-categorical reconstruction**: scikit-learn ships this dataset with wilderness area (4 values) and soil type (40 values) *already* one-hot encoded into 44 separate binary columns. We collapse them back into two compact categorical columns — a real, honest preprocessing step (not fabricated data) that both reduces dimensionality (54 → 12 columns) and gives every model in this series raw categorical columns to actually demonstrate native categorical handling on, rather than 44 pre-flattened binary features.

[ ]:
```
from sklearn.datasets import fetch_covtype

t0 = time.time()
covtype = fetch_covtype(as_frame=True, data_home=".cache_data").frame
print(f"Downloaded Covertype: {covtype.shape[0]:,} rows x {covtype.shape[1]}␣
 ↪columns in {time.time() - t0:.1f}s")
covtype.head()
```

1.4 3. Cleaning and Feature Engineering

[ ]:
```
df = covtype.copy()

wild_cols = [c for c in df.columns if c.startswith("Wilderness_Area_")]
soil_cols = [c for c in df.columns if c.startswith("Soil_Type_")]

# Reconstruct compact categorical columns from the one-hot encoding sklearn␣
 ↪ships this dataset
# with -- every row has exactly one wilderness area and one soil type set, so␣
 ↪argmax recovers the
# original category cleanly.
df["Wilderness_Area"] = df[wild_cols].to_numpy().argmax(axis=1).astype(str)
df["Soil_Type"] = df[soil_cols].to_numpy().argmax(axis=1).astype(str)
```

```
NUM_COLS = ["Elevation", "Aspect", "Slope", "Horizontal_Distance_To_Hydrology",
            "Vertical_Distance_To_Hydrology", "Horizontal_Distance_To_Roadways",
            "Hillshade_9am", "Hillshade_Noon", "Hillshade_3pm",␣
 ↪"Horizontal_Distance_To_Fire_Points"]
CAT_COLS = ["Wilderness_Area", "Soil_Type"]
TARGET = "Cover_Type"

for c in CAT_COLS:
    df[c] = df[c].astype("category")

X = df[NUM_COLS + CAT_COLS]
y = df[TARGET].astype(int)
print(f"Feature matrix: {X.shape[0]:,} rows x {X.shape[1]} columns "
      f"({len(NUM_COLS)} numeric, {len(CAT_COLS)} categorical: "
      f"Wilderness_Area has {df['Wilderness_Area'].nunique()} levels, "
      f"Soil_Type has {df['Soil_Type'].nunique()} levels)")
```

1.5 4. Exploratory Data Analysis

[ ]:
```
COVER_TYPE_NAMES = {
    1: "Spruce/Fir", 2: "Lodgepole Pine", 3: "Ponderosa Pine", 4: "Cottonwood/
 ↪Willow",
    5: "Aspen", 6: "Douglas-fir", 7: "Krummholz",
}

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

counts = y.value_counts().sort_index()
counts.index = [COVER_TYPE_NAMES[i] for i in counts.index]
counts.plot(kind="bar", ax=axes[0], color="#4C72B0")
axes[0].set_title("Cover type class balance")
axes[0].tick_params(axis="x", rotation=45)

axes[1].hist(X["Elevation"], bins=50, color="#55A868")
axes[1].set_title("Elevation distribution")
axes[1].set_xlabel("Elevation (m)")

df["Wilderness_Area"].value_counts().plot(kind="bar", ax=axes[2],␣
 ↪color="#C44E52")
axes[2].set_title("Wilderness area distribution")
axes[2].tick_params(axis="x", rotation=0)

plt.tight_layout()
plt.show()
```

```
print(f"Class imbalance ratio (largest / smallest): {counts.max() / counts.
 ↪min():.1f}x")
```

1.6 5. Train/Test Split and Context Size

`.fit()` on a `TabPFNClassifier` doesn't train weights — it registers your context rows for the model to read at prediction time. Given the GPU-or-bust reality described above, we size our context to what our own hardware can comfortably handle, rather than assuming we can push toward TabPFN-3's documented ceiling of up to 100,000+ rows.

[ ]:
```
X_train_pool, X_test_pool, y_train_pool, y_test_pool = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
print(f"Train pool: {len(X_train_pool):,} rows | Test pool: {len(X_test_pool):
 ↪,} rows")


def stratified_subsample(X, y, n, seed=42):
    idx, _ = train_test_split(np.arange(len(X)), train_size=min(n, len(X)),␣
 ↪stratify=y, random_state=seed)
    return X.iloc[idx].reset_index(drop=True), y.iloc[idx].
 ↪reset_index(drop=True)


N_EVAL = 3000
X_eval, y_eval = stratified_subsample(X_test_pool, y_test_pool, N_EVAL)
print(f"Eval set: {len(X_eval):,} rows (shared across every model we test)")
print(y_eval.value_counts(normalize=True).round(3))
```

[ ]:
```
N_CONTEXT = 8000 if DEVICE == "cuda" else 800
X_context, y_context = stratified_subsample(X_train_pool, y_train_pool,␣
 ↪N_CONTEXT)
print(f"Context: {len(X_context):,} rows")
```

1.7 6. A Quick Baseline

A plain `LogisticRegression` on the full training pool, for reference — the real head-to-head against Mitra and XGBoost lives in their own notebooks, evaluated on this exact same `X_eval`/`y_eval`.

[ ]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

logreg_pipe = Pipeline([
    ("prep", ColumnTransformer([
        ("num", StandardScaler(), NUM_COLS),
        ("cat", OneHotEncoder(handle_unknown="ignore"), CAT_COLS),
```

```
    ])),
    ("clf", LogisticRegression(max_iter=1000, random_state=42)),
])

t0 = time.time()
logreg_pipe.fit(X_train_pool, y_train_pool)
logreg_pred = logreg_pipe.predict(X_eval)
logreg_proba = logreg_pipe.predict_proba(X_eval)
print(f"LogisticRegression fit on {len(X_train_pool):,} rows in {time.time() -␣
 ↪t0:.1f}s")

results = {}


def score(name, y_true, pred, proba, classes):
    results[name] = {
        "accuracy": accuracy_score(y_true, pred),
        "balanced_accuracy": balanced_accuracy_score(y_true, pred),
        "f1_macro": f1_score(y_true, pred, average="macro"),
        "log_loss": log_loss(y_true, proba, labels=classes),
    }


score("LogisticRegression", y_eval, logreg_pred, logreg_proba, logreg_pipe.
 ↪classes_)
display(pd.DataFrame(results).T.round(4))
```

1.8 7. Zero-Shot Classification with TabPFN v3

Per TabPFN's own usage tips: *"calling `predict` on 100 samples separately is almost 100 times slower than a single call"* — so, as in every other notebook in this series, we batch our evaluation set and two synthetic "new patch" examples into one `predict_proba` call.

[ ]:
```
new_patches = pd.DataFrame([
    {"Elevation": 3200, "Aspect": 90, "Slope": 12,␣
 ↪"Horizontal_Distance_To_Hydrology": 150,
     "Vertical_Distance_To_Hydrology": 20, "Horizontal_Distance_To_Roadways":␣
 ↪800,
     "Hillshade_9am": 220, "Hillshade_Noon": 230, "Hillshade_3pm": 140,
     "Horizontal_Distance_To_Fire_Points": 900, "Wilderness_Area":␣
 ↪str(X_train_pool["Wilderness_Area"].mode()[0]),
     "Soil_Type": str(X_train_pool["Soil_Type"].mode()[0])},
    {"Elevation": 2400, "Aspect": 180, "Slope": 25,␣
 ↪"Horizontal_Distance_To_Hydrology": 60,
     "Vertical_Distance_To_Hydrology": 5, "Horizontal_Distance_To_Roadways":␣
 ↪300,
     "Hillshade_9am": 200, "Hillshade_Noon": 225, "Hillshade_3pm": 160,
```

```
    "Horizontal_Distance_To_Fire_Points": 400, "Wilderness_Area":␣
 ↪str(X_train_pool["Wilderness_Area"].mode()[0]),
    "Soil_Type": str(X_train_pool["Soil_Type"].mode()[0])},
])
for c_ in CAT_COLS:
    new_patches[c_] = new_patches[c_].astype("category")

X_eval_plus_new = pd.concat([X_eval, new_patches], ignore_index=True)
```

[ ]:
```
from tabpfn import TabPFNClassifier

clf = TabPFNClassifier(device=DEVICE, random_state=42)

t0 = time.time()
clf.fit(X_context, y_context.to_numpy())
proba_all = clf.predict_proba(X_eval_plus_new)
fit_time = time.time() - t0

proba_eval = proba_all[:len(X_eval)]
proba_new = proba_all[len(X_eval):]
pred_eval = clf.classes_[np.argmax(proba_eval, axis=1)]

print(f"TabPFN v3: {len(X_context):,} context rows + {len(X_eval_plus_new):,}␣
 ↪query rows "
      f"-> fit+predict in {fit_time:.1f}s")
score("TabPFN v3", y_eval, pred_eval, proba_eval, clf.classes_)
display(pd.DataFrame(results).T.round(4))
```

[ ]:
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_eval, pred_eval, labels=sorted(y.unique()))
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=[COVER_TYPE_NAMES[i] for i in sorted(y.unique())],
            yticklabels=[COVER_TYPE_NAMES[i] for i in sorted(y.unique())])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("TabPFN v3 -- confusion matrix")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

1.9 8. Interpreting a Black-Box In-Context Model

TabPFN has no per-dataset `.feature_importances_`. We use permutation importance on a small, dedicated context to keep this section's cost bounded.

[ ]:
```
PERM_FEATURES = ["Elevation", "Horizontal_Distance_To_Roadways", "Soil_Type",␣
 ↪"Wilderness_Area", "Slope"]

X_perm_context, y_perm_context = stratified_subsample(X_train_pool,␣
 ↪y_train_pool, 500, seed=7)
X_perm_eval, y_perm_eval = stratified_subsample(X_test_pool, y_test_pool, 150,␣
 ↪seed=7)

clf_perm = TabPFNClassifier(device=DEVICE, random_state=42)
t0 = time.time()
clf_perm.fit(X_perm_context, y_perm_context.to_numpy())


def acc_for(X_):
    p = clf_perm.predict_proba(X_)
    pred = clf_perm.classes_[np.argmax(p, axis=1)]
    return accuracy_score(y_perm_eval, pred)


base_acc = acc_for(X_perm_eval)
importances = {}
rng = np.random.RandomState(0)
for feat in PERM_FEATURES:
    X_shuffled = X_perm_eval.copy()
    X_shuffled[feat] = rng.permutation(X_shuffled[feat].to_numpy())
    importances[feat] = base_acc - acc_for(X_shuffled)

print(f"Micro-eval baseline accuracy: {base_acc:.3f}  ({time.time() - t0:.1f}s␣
 ↪total)")
pd.Series(importances).sort_values().plot(kind="barh", figsize=(6, 4),␣
 ↪color="#4C72B0",
                                          title="Permutation importance␣
 ↪(accuracy drop when shuffled)")
plt.xlabel("Accuracy drop")
plt.tight_layout()
plt.show()
```

1.10 9. Predictions for the New Patches

[ ]:
```
for i, row in new_patches.iterrows():
    top3_idx = np.argsort(proba_new[i])[::-1][:3]
    top3 = [(COVER_TYPE_NAMES[clf.classes_[j]], proba_new[i][j]) for j in␣
 ↪top3_idx]
    print(f"Patch {i + 1}: elevation={row['Elevation']}m,␣
 ↪slope={row['Slope']}°")
    print(f"  Top predictions: " + ", ".join(f"{name} ({p:.2f})" for name, p in␣
 ↪top3))
```

```
    print()
```

1.11 10. Practical Notes and Gotchas

- **GPU is close to mandatory.** TabPFN's own docs say CPU is realistic only under ~1,000 rows — we sized our context to `8000` on GPU and `800` on CPU specifically because of this documented constraint, not an arbitrary choice.
- **Batch your queries.** Per TabPFN's own usage tips, calling `predict` repeatedly on small batches is dramatically slower than one call with everything included — we fold our two new examples into the same call as the evaluation set for exactly this reason.
- **The weights are non-commercial licensed** (Prior Labs License) for the TabPFN-3 checkpoint specifically — check licensing before any commercial use.
- **7-class multiclass worked without any special configuration** — TabPFN handles multiclass natively.

[ ]:
```
display(pd.DataFrame(results).T.round(4))
print(f"\nTabPFN v3 wall-clock (fit+predict, {N_CONTEXT:,}-row context):␣
  ↪{fit_time:.1f}s")
```

1.12 11. Summary and References

TabPFN v3 handled a 7-class, imbalanced, 580,000-row dataset with a bounded context, using no hyperparameter tuning at all. Compare these numbers against the companion Mitra and XGBoost notebooks, evaluated on this exact same held-out set.

References:

- github.com/priorlabs/tabpfn — source code
- TabPFN-2.5 technical report (arXiv:2511.08667)
- Original Nature paper
- pypi.org/project/tabpfn
- Covertype dataset (UCI / scikit-learn)
- Blackard & Dean (1999), original dataset paper